RAG Assignment: Baseline vs. Re-Ranking

Report

Name: Avnish Uba Roll No: 2024115017

19 October 2026

GitHub Repo: `https://github.com/24KaratAu/lma-assignment-1`

Setup

All the code is in one notebook, `rag_baseline_vs_rerank.ipynb`, in the repo above.

- **Document:** Air India *Conditions of Carriage for Passengers, Baggage and Cargo* (27 pages). It is a legal contract with lots of rules, exceptions and "shall not" clauses, so it is good for testing retrieval.
- **Chunking:** 150 words per chunk with 30 words of overlap, which gave 105 chunks. I removed the page header and the table of contents first, because they would match almost every query.
- **Bi-Encoder:** `all-MiniLM-L6-v2`, with chunk vectors stored in a FAISS index (cosine similarity).
- **Cross-Encoder:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- **LLM:** Gemini 3.8 Flash at temperature 0. Both pipelines use the exact same prompt, so any difference in the answers comes only from which chunks were picked.

Pipeline 1 gives the bi-encoder's top 3 (out of its top 10) to the LLM. Pipeline 2 re-scores the same top 10 with the cross-encoder and gives its new top 3 to the LLM.

1 Architecture Comparison

The main difference is whether the query and the chunk are read separately or together.

Bi-Encoder: The query and each chunk go through the model separately, and each one becomes a single vector. Then the vectors are compared with cosine similarity. Since the chunk vectors do not depend on the query, they can be computed once and stored in FAISS. At search time only the query has to be embedded, so it is very fast. The problem is that a whole chunk is squeezed into one vector, and the query and chunk never actually "see" each other. Small but important words like "not", "except" or "domestic" can get lost, so a chunk that just shares words with the query can score high even when it is about something else.

Cross-Encoder: The query and the chunk are joined into one input (`[CLS] query [SEP] chunk [SEP]`) and go through the model together. Because of self-attention, every word in the query can look at every word in the chunk, and the model outputs one relevance score for the pair. This makes it much better with negation, conditions and exceptions. But nothing can be computed in advance, because the score depends on the query. Every (query, chunk) pair needs its own full pass through the model, which is slow.

2 Qualitative & Rank Shift Analysis

I wrote 30 test queries in three groups: keyword traps (KT), exceptions (EX) and negation (NG). For each query I marked the "gold" chunk, meaning the chunk that actually contains the answer. Out of 30 queries, the gold chunk was in the top 3 for **21** with the baseline and **26** with re-ranking. The table shows every query where re-ranking changed whether the gold chunk made the top 3.

ID	Query	Type	Gold chunk	Rank shift
KT3	What is the maximum compensation if a passenger dies on a domestic flight?	Keyword trap	78	#4 → #3
KT4	Can I cancel my ticket for free right after booking it?	Keyword trap	21	#5 → #1
EX3	Can I still get cancellation compensation if I gave a wrong phone number?	Exception	60	#4 → #1
NG3	Who is not allowed to change or waive the conditions of carriage?	Negation	104	#8 → #1
NG5	Does an unruly passenger have to pay if the plane is diverted?	Negation	71	#10 → #1
NG7	When will the airline not pay compensation for denied boarding?	Negation	35	#6 → #2
KT5	Is there an extra fee for booking on the Air India website?	Keyword trap	18	#3 → #6

In all six fixed cases the gold chunk was stuck at ranks 4–10, so the baseline never sent it to the LLM. NG5 is the biggest jump: the gold chunk was last in the top 10 and the cross-encoder moved it to first. For 3 queries (KT2, KT6, NG4) the gold chunk was not in the bi-encoder's top 10 at all, so re-ranking could not help. The cross-encoder can only re-order what stage 1 gives it.

Answer 1 (baseline) vs. Answer 2 (re-ranked)

The baseline did not really hallucinate, because the prompt tells the LLM to use only the given context. Instead it mostly **left out the answer** or gave the **wrong rule**, because the right chunk was outside the top 3.

- **KT3 (keyword trap):** The words "death" and "compensation" matched the international liability rules. Answer 1 only gave the international limits (128,821 SDRs) and said the context had nothing about domestic flights. Answer 2 gave the correct domestic limit of Rs. 25.5 lakh (Article 17.1.3).
- **KT4 (keyword trap):** Answer 1 only had the part of the rule saying you pay cancellation fees *after* 48 hours. It guessed from this that cancelling earlier is free, but it could not quote the actual rule. Answer 2 found the "look-in option" rule itself: you can cancel or change the ticket for free within 48 hours of booking, with the 7-day and 15-day conditions.
- **EX3 (exception):** Answer 1 said the context had no information. Answer 2 quoted the exception: a passenger is not eligible for compensation if they gave inadequate contact information when booking.
- **NG3 (negation):** Answer 1 said there was no information. Answer 2 quoted Article 20: no agent, employee or representative of the carrier can alter or waive the conditions.
- **NG5 (negation):** Answer 1 said the context did not cover it. Answer 2 quoted Article 12.3: if the plane is diverted to offload an unruly passenger, that passenger pays all the costs.

- **NG7 (negation):** Answer 1 talked about flight cancellations and unverified payments, which is the wrong topic. Answer 2 gave the correct rule: no compensation if the airline arranges another flight leaving within one hour of the original time.

Things that did not work perfectly:

- **KT5:** Re-ranking pushed the gold chunk down from #3 to #6. But Answer 2 was still correct (Rs. 300 convenience fee), because chunk 19 overlaps chunk 18 and also has the fee sentence. So re-ranking did not actually lose the answer here.
- **EX3:** The exception sentence continues with "provided, however…" into chunk 61, which did not make the top 3. Answer 2 even said the excerpt cuts off. This is a chunking problem: a fixed word count can split a rule from its condition.

3 Latency Profiling

All timings are on CPU, after a warm-up, averaged over 20 runs for each of the 30 queries (600 runs per pipeline). LLM time is measured separately because it is mostly network time and is the same kind of call in both pipelines.

Stage	Mean	Median	p95
Pipeline 1: Baseline retrieval (embed query + FAISS)	5.37 ms	5.06 ms	6.39 ms
Pipeline 2: Baseline + Cross-Encoder re-rank (10 pairs)	121.09 ms	118.05 ms	153.45 ms
Cross-Encoder step alone	116.46 ms	113.61 ms	148.09 ms
Gemini answer generation (14 calls)	6.25 s	5.88 s	–

Re-ranking makes retrieval about 22 times slower (5 ms to 121 ms). But compared to the LLM call, which takes about 6 seconds, it only adds around 2% to the total time, and in return the answers were much better. The FAISS search itself takes less than 0.1 ms; almost all of the baseline time is embedding the query.

4 Scaling Question

We do not run the cross-encoder on the whole collection because it has to do a full model pass for *every* chunk on *every* query. Nothing can be pre-computed, so the cost grows directly with the number of chunks. The bi-encoder does the expensive part (embedding the chunks) only once, offline, and at query time it just compares vectors.

To check this, I ran the cross-encoder on all 105 chunks for one query. It took 1.25 s, which is about 11.9 ms per pair. Using that number for bigger collections:

Number of chunks	Cross-Encoder on everything	Two-stage (top 10 + re-rank)
105 (this PDF)	1.25 s	0.11 s
10,000	119 s (about 2 min)	0.11 s
1,000,000	11,919 s (about 3.3 hours)	0.11 s
100,000,000	about 13.8 days	0.11 s

With the two-stage approach the cross-encoder always sees only 10 chunks, so its cost stays the same no matter how big the collection gets. The bi-encoder quickly narrows millions of chunks down to a few good candidates, and the cross-encoder only spends its time where accuracy matters.

One caveat: my notebook uses `IndexFlatIP`, which compares the query with every vector. This is fine for 105 chunks, but for millions of chunks an approximate index (like HNSW or IVF in FAISS) would be used to keep stage 1 fast too.